UNIVERSIDAD DE LAS FUERZAS ARMADAS-ESPE
SEDE SANTO DOMINGO DE LOS TSÁCHILAS

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN - DCCO-SS

CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

PERIODO	**:**	202650 abril 2026 – agosto 2026
ASIGNATURA	**:**	Modelos discretos para la ing
TEMA	**:**	LEYES DE LA LÓGICA PROPOSICIONAL
ESTUDIANTE	**:**	Jeffry Vera, Darlin Demera
NIVEL-PARALELO - NRC:		3vo ITIN
DOCENTE	**:**	Javier José Cevallos Farías
FECHA DE ENTREGA	**:**	<26/04/2026>

SANTO DOMINGO - ECUADOR

Contenido

INTRODUCCION 3

OBJETIVOS 5

Objetivo General 5

Objetivos Específicos 5

DESARROLLO 6

LEY DISTRIBUTIVA 6

LEY DE DE MORGAN 9

LEY DE ABSORCIÓN 12

LEY DE COMPLEMENTO 14

IMPLICACIÓN (CONDICIONAL) 17

BICONDICIONAL 20

CONCLUSIONES 24

RECOMENDACIONES 24

REFERENCIAS 25

ANEXOS 26

INTRODUCCION

La lógica proposicional representa uno de los pilares fundamentales tanto de la matemática discreta como de la ciencia de la computación moderna. Su objeto de estudio son las proposiciones —enunciados declarativos que admiten exclusivamente los valores de verdad (V) o falso (F)— y las relaciones que se establecen entre ellas mediante conectivos lógicos: la conjunción (∧), la disyunción (∨), la negación (¬), la implicación (→) y el bicondicional (↔) *(Rosen, 2019; Mendelson, 2015)*.

Dentro de este campo, las leyes de equivalencia lógica ocupan un lugar central. Estas leyes posibilitan la transformación y simplificación de expresiones proposicionales sin alterar su valor de verdad: dos fórmulas son lógicamente equivalentes cuando producen resultados idénticos en todas las combinaciones posibles de sus variables. Cuando una equivalencia es siempre verdadera, recibe el nombre de tautología; cuando es siempre falsa, se denomina contradicción *(Grimaldi, 2004)*. Estas propiedades son fundamentales para demostrar teoremas, diseñar circuitos digitales y optimizar algoritmos de programación.

El presente informe examina en profundidad seis de las leyes de equivalencia lógica de mayor relevancia teórica y práctica: la Ley Distributiva, la Ley de De Morgan, la Ley de Absorción, la Ley de Complemento, la Implicación y el Bicondicional. Para cada ley se proporciona su definición formal, su expresión simbólica, una explicación detallada de su funcionamiento, ejemplos verificados mediante tablas de verdad y aplicaciones directas en el ámbito de la informática y la programación *(Johnsonbaugh, 2005)*.

Importancia en la Lógica y la Programación

Las leyes de equivalencia lógica trascienden el ámbito puramente teórico. En la práctica profesional, poseen aplicaciones directas en múltiples áreas de la informática *(Mano & Ciletti, 2013)***:**

- Diseño de circuitos digitales: Las compuertas lógicas (AND, OR, NOT) implementan físicamente las operaciones proposicionales. La simplificación de expresiones booleanas reduce el número de compuertas necesarias, disminuyendo costos de fabricación y consumo energético (Mano & Ciletti, 2013).
- Optimización de condiciones en código fuente: Los compiladores y desarrolladores aplican las leyes de De Morgan para invertir y simplificar condiciones complejas, mejorando la legibilidad y reduciendo la probabilidad de introducir errores en estructuras condicionales.
- Bases de datos y consultas SQL: Las cláusulas WHERE utilizan lógica proposicional directamente; la simplificación de condiciones booleanas puede mejorar significativamente el rendimiento de las consultas y la claridad del código SQL.
- Inteligencia artificial y sistemas expertos: Los motores de inferencia lógica se fundamentan en estas leyes para deducir conclusiones válidas a partir de conjuntos de premisas (Mendelson, 2015; Stanford Encyclopedia of Philosophy, 2021).
- Verificación formal de software: La comprobación automática de corrección en programas se basa en demostrar equivalencias entre especificaciones formales y el comportamiento real del código.

OBJETIVOS

Objetivo General

Analizar y aplicar las principales leyes de equivalencia lógica de la lógica proposicional —Ley Distributiva, Ley de De Morgan, Ley de Absorción, Ley de Complemento, Implicación y Bicondicional— mediante el estudio teórico, la demostración con tablas de verdad y la identificación de sus aplicaciones en el campo de la informática y la programación.

Objetivos Específicos

- Definir y expresar formalmente cada ley de equivalencia lógica, identificando su estructura simbólica y las condiciones bajo las cuales se aplica correctamente, a fin de consolidar los fundamentos teóricos de la lógica proposicional.
- Verificar la validez de cada ley de equivalencia a través de la construcción de tablas de verdad completas, demostrando matemáticamente que las expresiones equivalentes producen resultados idénticos en todos los casos posibles.
- Relacionar las leyes de la lógica proposicional con problemas prácticos del ámbito computacional, como la simplificación de condiciones en código fuente, el diseño de circuitos lógicos y la optimización de consultas en bases de datos, evidenciando la aplicabilidad directa de estos conceptos.

DESARROLLO

LEY DISTRIBUTIVA

Definición

La Ley Distributiva establece que una operación lógica puede distribuirse sobre otra de manera análoga a la propiedad distributiva del álgebra aritmética. Esta ley presenta dos formas simétricas: la distribución de la conjunción (∧) sobre la disyunción (∨) y la distribución de la disyunción (∨) sobre la conjunción (∧). En ambos casos, la expansión o factorización de la expresión produce una fórmula lógicamente equivalente, es decir, con idéntico valor de verdad en todas las interpretaciones posibles *(Grimaldi, 2004, p. 47; Lipschutz & Lipson, 2007)*.

Esta ley es fundamental en el álgebra de Boole —el sistema algebraico sobre el que se sustenta la electrónica digital— y permite tanto expandir expresiones complejas para facilitar su análisis como factorizarlas para reducir su complejidad computacional *(Mano & Ciletti, 2013)*.

Fórmulas Lógicas

Distributiva de ∧ sobre ∨: $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$
Distributiva de ∨ sobre ∧: $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$

Explicación Paso a Paso

Se trabaja con tres proposiciones: p, q, r. Para verificar la primera forma:

- Paso 1: Calcular q ∨ r — es verdadera cuando al menos uno de q o r es verdadero.

- Paso 2: Calcular p ∧ (q ∨ r) — es verdadera solo cuando p es verdadero y (q ∨ r) también lo es.
- Paso 3: Calcular p ∧ q y p ∧ r por separado.
- Paso 4: Calcular (p ∧ q) ∨ (p ∧ r) — es verdadera cuando al menos uno de los dos productos es verdadero.
- Paso 5: Comparar columnas — si los resultados de los pasos 2 y 4 son idénticos en todas las filas, la ley queda demostrada.

Ejemplo Resuelto

Verificar que: p ∧ (q ∨ r) ≡ (p ∧ q) ∨ (p ∧ r)

Tabla de Verdad — Ley Distributiva: p ∧ (q ∨ r) ≡ (p ∧ q) ∨ (p ∧ r)

p	q	r	q ∨ r	p ∧ (q ∨ r)	p ∧ q	p ∧ r	(p∧q)∨(p∧r)
V	V	V	V	V	V	V	**V**
V	V	F	V	V	V	F	**V**
V	F	V	V	V	F	V	**V**
V	F	F	F	F	F	F	**F**
F	V	V	V	F	F	F	**F**
F	V	F	V	F	F	F	**F**

p	q	r	q ∨ r	p ∧ (q ∨ r)	p ∧ q	p ∧ r	(p∧q)∨(p∧r)
F	F	V	V	F	F	F	**F**
F	F	F	F	F	F	F	**F**

✓ Conclusión: Las columnas 'p ∧ (q ∨ r)' y '(p ∧ q) ∨ (p ∧ r)' son idénticas en todas las combinaciones. La Ley Distributiva queda demostrada.

Aplicación en Informática / Programación

En programación, la Ley Distributiva se usa para refactorizar condiciones booleanas complejas. Considérese el siguiente código en Java:

```
// Forma original (sin distribuir)
if (usuario && (esAdmin || esSupervisor)) {
    accederPanel();
}
// Forma distribuida (equivalente)
if ((usuario && esAdmin) || (usuario && esSupervisor)) {
    accederPanel();
}
```

Ambas formas son lógicamente equivalentes. La segunda forma puede ser útil cuando se necesita agregar lógica diferente para cada rol en bloques separados.

LEY DE DE MORGAN

Definición

Las Leyes de De Morgan, formuladas por el matemático y lógico británico Augustus De Morgan (1806–1871), describen el comportamiento de la negación cuando se aplica sobre una conjunción o una disyunción compuestas. En términos intuitivos: la negación de una conjunción (AND) equivale a la disyunción (OR) de las negaciones individuales de cada proposición, y la negación de una disyunción (OR) equivale a la conjunción (AND) de las negaciones individuales *(Mendelson, 2015; García Merayo, 2009)*.

Estas leyes son, posiblemente, las más empleadas en la práctica del desarrollo de software, dado que los programadores frecuentemente necesitan invertir condiciones compuestas para construir su complemento lógico sin incurrir en errores semánticos *(Rosen, 2019, p. 28)*.

Fórmulas Lógicas

Primera Ley: $\neg(p \wedge q) \equiv \neg p \vee \neg q$
Segunda Ley: $\neg(p \vee q) \equiv \neg p \wedge \neg q$

Explicación Paso a Paso

Para la Primera Ley de De Morgan:

- Paso 1: Calcular $p \wedge q$ — verdadera solo cuando ambas son verdaderas.
- Paso 2: Negar el resultado $\rightarrow \neg(p \wedge q)$ — falsa solo cuando ambas eran verdaderas.

- Paso 3: Calcular ¬p y ¬q por separado (negar cada proposición individualmente).
- Paso 4: Calcular ¬p ∨ ¬q — verdadera cuando al menos una de las negaciones es verdadera.
- Paso 5: Verificar que las columnas de los pasos 2 y 4 sean idénticas — si lo son, la ley queda demostrada.

Ejemplo Resuelto

Verificar que: ¬(p ∧ q) ≡ ¬p ∨ ¬q (Primera Ley de De Morgan)

Tabla de Verdad — Primera Ley de De Morgan: ¬(p ∧ q) ≡ ¬p ∨ ¬q

p	q	p ∧ q	¬(p ∧ q)	¬p	¬q	¬p ∨ ¬q
V	V	V	F	F	F	**F**
V	F	F	V	F	V	**V**
F	V	F	V	V	F	**V**
F	F	F	V	V	V	**V**

✓ Conclusión: Las columnas '¬(p ∧ q)' y '¬p ∨ ¬q' son idénticas. La Primera Ley de De Morgan queda demostrada.

Aplicación en Informática / Programación

La Ley de De Morgan es la más aplicada en el desarrollo de software. En Java, se usa constantemente para invertir condiciones en estructuras condicionales:

```
// ¬(p ∧ q): Negar una condición AND
if (!(esMayorDeEdad && tienePermiso)) { ... }
// Equivalente por De Morgan:
if (!esMayorDeEdad || !tienePermiso) { ... }


// ¬(p ∨ q): Negar una condición OR
if (!(esBloqueado || estaSuspendido)) { ... }
// Equivalente por De Morgan:
if (!esBloqueado && !estaSuspendido) { ... }
```

Esta transformación mejora la legibilidad del código y es especialmente útil en revisiones de código y refactorizaciones. Los linters modernos como SonarQube sugieren aplicar De Morgan automáticamente para simplificar condiciones negadas.

LEY DE ABSORCIÓN

Definición

La Ley de Absorción establece que cuando una proposición se combina mediante una operación lógica con una expresión que ya la contiene como parte de su estructura, la proposición 'absorbe' la expresión compuesta, y el resultado se simplifica al valor de la proposición original. En otras palabras, la presencia de la variable en ambos operandos hace que la parte más compleja resulte redundante y pueda eliminarse sin modificar el valor de la expresión *(Rosen, 2019; Lipschutz & Lipson, 2007, p. 18)*.

Esta ley es particularmente valiosa en la simplificación de expresiones booleanas en álgebra de Boole y en la eliminación de condiciones redundantes en código fuente.

Fórmulas Lógicas

Primera forma (∧): $p \wedge (p \vee q) \equiv p$
Segunda forma (∨): $p \vee (p \wedge q) \equiv p$

Explicación Paso a Paso

Para la primera forma: $p \wedge (p \vee q) \equiv p$

- Paso 1: Evaluar $p \vee q$ — verdadera cuando al menos uno de los dos es verdadero.
- Paso 2: Evaluar $p \wedge (p \vee q)$ — si p es falsa, el AND siempre es falso. Si p es verdadera, $(p \vee q)$ también lo es (pues p aparece en ella), así que el AND es verdadero.

- Paso 3: Por lo tanto, el resultado siempre coincide exactamente con el valor de p.
- Paso 4: Verificar con la tabla de verdad que la columna 'p ∧ (p ∨ q)' sea idéntica a la columna 'p'.

Ejemplo Resuelto

Verificar que: $p \wedge (p \vee q) \equiv p$ (Primera Ley de Absorción)

Tabla de Verdad — Ley de Absorción: p ∧ (p ∨ q) ≡ p

p	q	p ∨ q	p ∧ (p ∨ q)	¿ = p?
V	V	V	V	✓
V	F	V	V	✓
F	V	V	F	✓
F	F	F	F	✓

✓ Conclusión: La columna 'p ∧ (p ∨ q)' es idéntica a 'p'. La expresión compleja queda 'absorbida' por p. Ley demostrada.

Aplicación en Informática / Programación

En programación, la Ley de Absorción permite detectar condiciones redundantes que consumen tiempo de procesamiento sin aportar información adicional:

```
// Condición redundante (antes de simplificar)
boolean activo = true;
if (activo && (activo || tieneSuscripcion)) {
    mostrarContenido();
}
// Simplificado por Absorción: p ∧ (p ∨ q) ≡ p
if (activo) { mostrarContenido(); }
```

Eliminar condiciones redundantes no solo hace el código más limpio, sino que también mejora el rendimiento al evitar evaluaciones innecesarias en cada iteración.

LEY DE COMPLEMENTO

Definición

Las Leyes de Complemento describen el comportamiento de una proposición cuando se combina con su propia negación mediante los conectivos AND y OR. Estas leyes son fundamentales porque definen los valores extremos del sistema lógico binario y son la base de las nociones de contradicción y tautología, dos conceptos esenciales en lógica formal, demostración matemática y diseño de sistemas digitales *(Mendelson, 2015; Rosen, 2019, p. 25)*.

Se distinguen cuatro formas, incluyendo la doble negación: cuando una proposición se combina con su negación mediante AND, el resultado es siempre falso (contradicción); mediante OR, siempre verdadero (tautología); y la negación de la negación devuelve la proposición original.

Fórmulas Lógicas

Complemento ∧: $p \wedge \neg p \equiv F$ (Contradicción)
Complemento ∨: $p \vee \neg p \equiv V$ (Tautología)
Doble Negación: $\neg(\neg p) \equiv p$

Explicación Paso a Paso

Para $p \wedge \neg p$ (Contradicción):

- Paso 1: Para cualquier valor de p, ¬p tiene el valor contrario.
- Paso 2: Una conjunción (AND) requiere que ambos operandos sean verdaderos.
- Paso 3: Como p y ¬p siempre tienen valores opuestos, nunca pueden ser ambos verdaderos simultáneamente.
- Paso 4: Por lo tanto, $p \wedge \neg p$ es siempre FALSO — es una contradicción.

Para $p \vee \neg p$ (Tautología):

- Paso 1: La disyunción (OR) es verdadera cuando al menos un operando es verdadero.
- Paso 2: Como p y ¬p siempre tienen valores opuestos, siempre habrá al menos uno verdadero.
- Paso 3: Por lo tanto, $p \vee \neg p$ es siempre VERDADERO — es una tautología.

Ejemplo Resuelto

Verificar: $p \wedge \neg p \equiv F$ y $p \vee \neg p \equiv V$

Tabla de Verdad — Ley de Complemento

p	¬p	p ∧ ¬p	p ∨ ¬p
V	F	F	**V**
F	V	F	**V**

✓ Conclusión: 'p ∧ ¬p' es siempre F (contradicción), y 'p ∨ ¬p' es siempre V (tautología). Ambas leyes de complemento quedan demostradas.

Aplicación en Informática / Programación

Las Leyes de Complemento tienen aplicaciones directas en la detección de errores lógicos en código. Un programador que escribe una condición del tipo (x > 5 && x <= 5) está creando una contradicción que nunca se cumple:

```
// CONTRADICCIÓN — este bloque NUNCA se ejecuta
if (conectado && !conectado) {
    iniciarSesion(); // código muerto
}
```

```
// TAUTOLOGÍA — este bloque SIEMPRE se ejecuta
if (conectado || !conectado) {
    log('Esto siempre ocurre');
}
```

Los analizadores estáticos de código (como PMD, Checkstyle o SonarQube) detectan automáticamente contradicciones y tautologías, alertando al desarrollador sobre posibles errores de lógica.

IMPLICACIÓN (CONDICIONAL)

Definición

La implicación lógica, también denominada condicional, expresa una relación de tipo causa-consecuencia entre dos proposiciones: 'si p, entonces q', representada simbólicamente como $p \rightarrow q$. La proposición p recibe el nombre de antecedente (o hipótesis), y q el de consecuente (o conclusión). El condicional es FALSO exclusivamente cuando el antecedente es verdadero y el consecuente es falso; en todas las demás combinaciones es verdadero, incluido el caso en que el antecedente es falso —situación en la que se habla de 'verdad vacua' o ex falso quodlibet *(Mendelson, 2015; Stanford Encyclopedia of Philosophy, 2021)*.

La equivalencia fundamental $p \rightarrow q \equiv \neg p \vee q$ permite expresar el condicional en términos de negación y disyunción, operaciones más básicas del sistema lógico, lo cual resulta esencial para la simplificación de expresiones en álgebra de Boole *(Rosen, 2019, p. 29)*.

Fórmulas Lógicas

Condicional: $p \rightarrow q \equiv \neg p \vee q$
Contrarrecíproca: $\neg q \rightarrow \neg p \equiv p \rightarrow q$
Contraposición: $p \rightarrow q \equiv \neg q \rightarrow \neg p$

Explicación Paso a Paso

Para entender cuándo la implicación es falsa:

- Paso 1: Identificar antecedente (p) y consecuente (q).
- Paso 2: Si p = V y q = V: la promesa se cumple → VERDADERO.
- Paso 3: Si p = V y q = F: la promesa se rompe → FALSO (único caso falso).
- Paso 4: Si p = F y q = V: la condición no se activó pero el resultado es bueno → VERDADERO.
- Paso 5: Si p = F y q = F: la condición no se activó → VERDADERO (vacuamente).

Analogía: 'Si estudio (p), apruebo (q)'. Solo fallo en mi promesa si estudié y no aprobé.

Ejemplo Resuelto

Verificar que: $p \rightarrow q \equiv \neg p \vee q$

Tabla de Verdad — Implicación: $p \rightarrow q \equiv \neg p \vee q$

p	q	p → q	¬p	¬p ∨ q	¿ ≡ ?
V	V	V	F	V	✓
V	F	F	F	F	✓
F	V	V	V	V	✓
F	F	V	V	V	✓

✓ Conclusión: Las columnas 'p → q' y '¬p ∨ q' son idénticas. La equivalencia de la implicación queda demostrada. La implicación solo es FALSA cuando p=V y q=F.

Aplicación en Informática / Programación

En programación, la implicación se traduce directamente en estructuras condicionales. El operador ternario y las estructuras if-else modelan exactamente el condicional lógico:

```
// Implicación: Si tiene sesión (p) → mostrar dashboard (q)
if (tieneSesion) {
    mostrarDashboard();
}
// Contraposición: ¬q → ¬p
// Si no puede mostrar dashboard → no tiene sesión válida
if (!puedeCargarDashboard) {
    cerrarSesion();
```

```
    }
```

La contrarrecíproca (¬q → ¬p ≡ p → q) es ampliamente utilizada en demostraciones matemáticas y en el diseño de pruebas unitarias: si una función no devuelve el resultado esperado (¬q), podemos deducir que la entrada no cumplió la condición requerida (¬p).

BICONDICIONAL

Definición

El bicondicional, también denominado doble implicación o equivalencia lógica, se representa como p ↔ q y se lee 'p si y solo si q'. Esta operación establece que las dos proposiciones tienen exactamente el mismo valor de verdad en toda circunstancia: son ambas verdaderas o ambas falsas de manera simultánea. Se trata del conectivo más restrictivo, pues el único patrón de verdad que acepta es la coincidencia total de valores entre sus operandos *(Mendelson, 2015; Grimaldi, 2004, p. 52)*.

El bicondicional es equivalente a la conjunción de ambas implicaciones mutuas: (p → q) ∧ (q → p), y también a la expresión (p ∧ q) ∨ (¬p ∧ ¬q). Su negación, ¬(p ↔ q), corresponde a la disyunción exclusiva (XOR), ampliamente empleada en criptografía y en circuitos digitales *(Mano & Ciletti, 2013; Rosen, 2019, p. 30)*.

Fórmulas Lógicas

Bicondicional: p ↔ q ≡ (p → q) ∧ (q → p)
Equivalente: p ↔ q ≡ (p ∧ q) ∨ (¬p ∧ ¬q)

Negación: $\neg(p \leftrightarrow q) \equiv p \oplus q$ **(XOR)**

Explicación Paso a Paso

Para verificar $p \leftrightarrow q \equiv (p \rightarrow q) \land (q \rightarrow p)$:

- Paso 1: Calcular $p \rightarrow q$ (implicación directa).
- Paso 2: Calcular $q \rightarrow p$ (implicación inversa o recíproca).
- Paso 3: Calcular $(p \rightarrow q) \land (q \rightarrow p)$ — verdadera cuando ambas implicaciones son verdaderas.
- Paso 4: Calcular $p \leftrightarrow q$ directamente — verdadera cuando p y q tienen el mismo valor.
- Paso 5: Verificar que las columnas '$p \leftrightarrow q$' y '$(p \rightarrow q) \land (q \rightarrow p)$' sean idénticas.

Analogía: 'Apruebo el examen (p) si y solo si estudio lo suficiente (q)'. Significa que si estudio apruebo, Y si apruebo es porque estudié.

Ejemplo Resuelto

Verificar: $p \leftrightarrow q \equiv (p \rightarrow q) \land (q \rightarrow p)$

Tabla de Verdad — Bicondicional: $p \leftrightarrow q \equiv (p \rightarrow q) \land (q \rightarrow p)$

p	q	p → q	q → p	(p→q)∧(q→p)	p ↔ q	¿ ≡ ?
V	V	V	V	V	V	✓
V	F	F	V	F	F	✓
F	V	V	F	F	F	✓
F	F	V	V	V	V	✓

✓ Conclusión: Las columnas 'p ↔ q' y '(p → q) ∧ (q → p)' son idénticas. El bicondicional es verdadero solo cuando p y q tienen el mismo valor de verdad.

Aplicación en Informática / Programación

El bicondicional modela la igualdad lógica perfecta entre condiciones. En programación, el equivalente más directo es el operador de igualdad (==) para booleanos o el operador XNOR en hardware digital:

```
// Bicondicional: acceso si y solo si (admin && activo)
boolean p = esAdmin;
boolean q = cuentaActiva;
boolean bicondicional = (p == q); // V solo si ambos iguales


// En validaciones doble sentido:
```

```
// El token es válido SI Y SOLO SI el usuario existe
if ((token != null) == (usuario != null)) {
    procesarSolicitud();
}
```

En bases de datos, el bicondicional se usa en joins y correlaciones: 'un registro de ventas corresponde a un cliente si y solo si el ID del cliente en ventas coincide con el ID en la tabla de clientes' (modelo relacional). En criptografía, los cifrados de bloque utilizan operaciones XOR (negación del bicondicional) para garantizar que solo quien conoce la clave pueda descifrar el mensaje.

CONCLUSIONES

- Las leyes de equivalencia lógica —Distributiva, De Morgan, Absorción, Complemento, Implicación y Bicondicional— constituyen un sistema coherente y formalmente completo de transformaciones que permiten manipular expresiones proposicionales sin alterar su semántica. Su estudio pone de manifiesto que la lógica formal no es un conjunto de reglas arbitrarias, sino que obedece a patrones estructurales precisos y verificables, aplicables tanto en demostraciones matemáticas como en el desarrollo tecnológico

- La verificación mediante tablas de verdad demostró ser un método poderoso, sistemático y exhaustivo para validar equivalencias lógicas. Al evaluar todas las combinaciones posibles de valores de las proposiciones involucradas, se garantiza que la equivalencia tiene carácter universal y no depende de casos particulares. Este enfoque constituye el fundamento de la verificación formal de software y del diseño automatizado de circuitos digitales

RECOMENDACIONES

Se recomienda construir tablas de verdad de forma manual en las etapas iniciales del aprendizaje, antes de recurrir a herramientas de verificación automática. Este proceso desarrolla la comprensión intuitiva del funcionamiento de los conectivos lógicos y fomenta la capacidad de detectar errores en expresiones proposicionales complejas durante el desarrollo de software *(Grimaldi, 2004)*.

Desde el punto de vista técnico, se sugiere incorporar las Leyes de De Morgan como un heurístico inmediato en las revisiones de código (*code review*). Al identificar una negación

aplicada sobre un operador AND u OR, la transformación por De Morgan debe convertirse en un reflejo automático del programador, mejorando tanto la legibilidad como la corrección lógica del código *(GeeksForGeeks, 2023; Johnsonbaugh, 2005)*.

REFERENCIAS

García Merayo, F. (2009). Matemática discreta. Ediciones Paraninfo.

GeeksForGeeks. (2023). Mathematical logic – Laws of Boolean algebra. https://www.geeksforgeeks.org/mathematical-logic-laws-of-boolean-algebra/

Grimaldi, R. P. (2004). Matemáticas discretas y combinatoria (4.ª ed.). Pearson Educación.

Johnsonbaugh, R. (2005). Matemáticas discretas (6.ª ed.). Pearson Prentice Hall.

Lipschutz, S., & Lipson, M. (2007). Matemáticas discretas (Serie Schaum, 3.ª ed.). McGraw-Hill.

Mano, M. M., & Ciletti, M. D. (2013). Digital design: With an introduction to the Verilog HDL (5.ª ed.). Pearson.

Mendelson, E. (2015). Introduction to mathematical logic (6.ª ed.). CRC Press / Taylor & Francis Group.

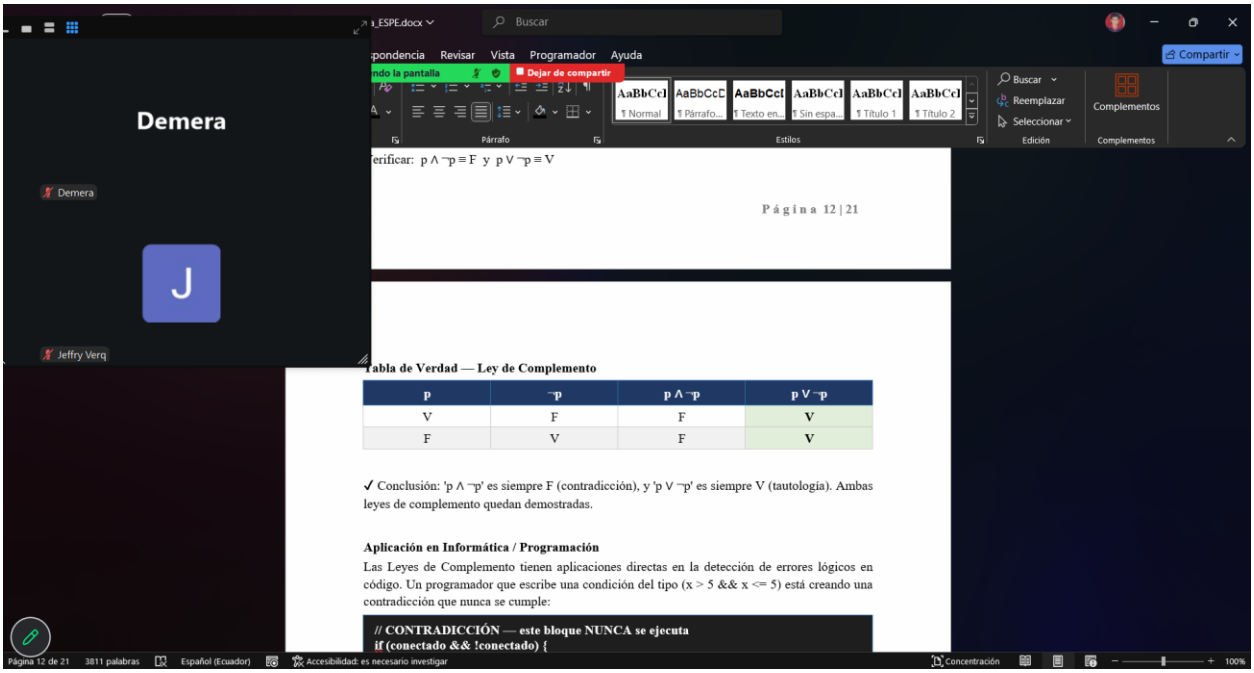
Rosen, K. H. (2019). Discrete mathematics and its applications (8.ª ed.). McGraw-Hill Education.

Stanford Encyclopedia of Philosophy. (2021). Classical logic. The Metaphysics Research Lab, Stanford University. https://plato.stanford.edu/entries/logic-classical/

Wolfram MathWorld. (2024). Propositional calculus. https://mathworld.wolfram.com/PropositionalCalculus.html

ANEXOS

Imagen de nosotros asiendo el deber





Integrante	Tarea Asignada	Estado
Integrante 1	Investigación bibliográfica + Introducción y Objetivos	✓ Completado
Integrante 2	Ley Distributiva y Ley de De Morgan + Tablas de verdad	✓ Completado
Integrante 1	Ley de Absorción y Complemento + Tablas de verdad	✓ Completado
Integrante 2	Implicación y Bicondicional + Aplicaciones en programación	✓ Completado